Dominio 5: gestione del contesto e affidabilità (15%)

Questo dominio tratta la gestione del contesto della conversazione, i modelli di escalation, la propagazione degli errori, le strategie per gestire il contesto di codebase ampie, i workflow di revisione umana e la provenienza delle informazioni.

5.1 Gestione del contesto della conversazione

La finestra di contesto è limitata

Claude ha una finestra di contesto limitata. Ogni messaggio, risultato di uno strumento e prompt di sistema consuma token. Nelle conversazioni lunghe, le prestazioni possono peggiorare perché le informazioni precedenti vengono compresse o perse.

Dimensioni attuali: «1 milione di token» cambia la natura del problema, ma non lo elimina.

Modello	Finestra di contesto	Output massimo
Claude Fable 5.1 / Fable 5 / Opus 5.5 / Opus 5 / Sonnet 5	1M tokens	128k
Claude Opus 4.8 / 4.7 / 4.6, Sonnet 4.6	1M tokens	128k
Claude Haiku 4.5	200k tokens	64k

Una finestra da 1 milione di token comporta due rischi:

1. **Una finestra più ampia non è gratuita.** Ogni token viene addebitato a ogni turno e l'effetto «perso nel mezzo», descritto sotto, peggiora *all'aumentare* della lunghezza. Riempire una finestra da 1 milione di token solo perché è possibile aumenta i costi e riduce la qualità.
2. **Il conteggio dei token è cambiato.** Opus 4.7 ha introdotto un nuovo tokenizer, adottato anche da Opus 4.8, Opus 5, Opus 5.5, Sonnet 5 e Fable 5 / 5.1: lo stesso testo produce circa il 30% di token in più rispetto ai modelli precedenti a 4.7 (oggi 1 milione di token corrisponde a circa 555.000 parole, contro circa 750.000 in precedenza). Ricalibra budget, dimensioni dei blocchi e soglie definiti per i modelli precedenti usando `messages.count_tokens`, mai `tiktoken`.

Rischi della sintesi progressiva

Quando il contesto viene compresso (con `/compact` o tramite compressione automatica), il sistema riassume la conversazione precedente. Questo comporta alcuni rischi:

Cosa può andare perso nel riepilogo:

- Valori numerici esatti (`$149.99` diventa «circa $150»).
- Date precise (`2024-01-15` diventa «metà gennaio»).
- Percentuali esatte (`23.7%` diventa «circa il 24%»).
- Numeri d'ordine, ID cliente e codici di riferimento.
- Messaggi di errore e stack trace esatti.

Soluzione: blocco dei fatti del caso

Estrai i dati transazionali fondamentali in un blocco strutturato e persistente, che sopravviva ai riepiloghi:

```
FATTI DEL CASO (non riassumere):
- ID cliente: n. 12345
- Ordine: n. 67890
- Data ordine: 2024-01-15
- Totale: $149.99
- Problema: prodotto danneggiato alla consegna
- Rimborso richiesto: rimborso completo ($149.99)
```

Effetto «perso nel mezzo»

I modelli elaborano **l'inizio** e **la fine** di input lunghi in modo più affidabile rispetto alla **parte centrale**. Le informazioni collocate nel mezzo di un contesto ampio hanno maggiori probabilità di essere trascurate.

Strategie di mitigazione:

1. Inserisci i riepiloghi dei risultati principali **all'inizio** degli input aggregati.
2. Usa intestazioni di sezione esplicite per creare una struttura di navigazione.
3. Evita di collocare informazioni critiche nel mezzo di documenti lunghi.
4. Se devi includere molto contenuto, posiziona le informazioni importanti sia all'inizio sia alla fine.

Riduzione dell'output degli strumenti

I risultati degli strumenti possono essere prolissi. Una ricerca di un ordine cliente potrebbe restituire 40 campi quando te ne servono solo 5.

Da evitare: restituire tutti e 40 i campi, consumando inutilmente contesto.

Corretto: limita i risultati ai campi pertinenti prima di restituirli a Claude:

```
{
  "order_id": "67890",
  "status": "delivered",
  "delivery_date": "2024-01-20",
  "total": 149.99,
  "items": ["Widget Pro (x2)"]
}
```

Cosa sopravvive a /compact

Sopravvive	Può andare perso
CLAUDE.md (riletto dal disco)	Dettagli delle conversazioni precedenti
Prompt di sistema	Risultati specifici degli strumenti
Messaggi recenti	Ragionamenti sfumati dei turni precedenti

Sopravvive	Può andare perso
Memoria automatica (MEMORY.md)	Citazioni e numeri esatti del contesto iniziale

5.2 Modelli di escalation

Quando effettuare subito un'escalation

Condizione	Motivo	Azione
Il cliente chiede esplicitamente di parlare con una persona	Rispetto dell'autonomia dell'utente	Effettua subito l'escalation, senza ulteriori indagini
Lacuna o eccezione nelle policy	L'agente non può prendere decisioni di policy	Effettua l'escalation con un riepilogo strutturato del contesto
Nessun progresso dopo tentativi ragionevoli	L'agente è bloccato	Effettua l'escalation con un riepilogo dei tentativi fatti

Quando NON effettuare un'escalation

Condizione inadeguata	Perché non è valida
Emozione o tono molto acceso	Emozione ≠ complessità. Un cliente arrabbiato può avere un problema semplice.
Bassa affidabilità dichiarata dal modello	I punteggi di affidabilità di Claude sono calibrati male. Una risposta con «bassa affidabilità» potrebbe essere corretta, mentre una con «alta affidabilità» potrebbe essere errata.
Conversazione lunga	La durata non indica la necessità di un'escalation. Alcuni problemi richiedono legittimamente più turni.

Distinzione tra sentiment e complessità

È un concetto verificato esplicitamente nell'esame:

```
"SONO DAVVERO FRUSTRATO!!!" + semplice errore di fatturazione
→ L'agente può risolverlo. Non effettuare l'escalation in base alle maiuscole e ai
punti esclamativi.

"Ho una domanda sull'interazione tra il vostro SLA aziendale
e i nuovi requisiti UE sulla residenza dei dati" + tono calmo
→ Potrebbe essere necessaria un'escalation per la complessità delle policy,
nonostante il tono tranquillo.
```

Più corrispondenze per il cliente

Quando la ricerca di un cliente restituisce più corrispondenze:

Errato: scegliere la corrispondenza più probabile usando euristiche. **Corretto:** chiedere al cliente altre informazioni identificative per distinguere gli account.

```
"Ho trovato più account con questo nome. Puoi indicarmi
il tuo indirizzo email o le ultime quattro cifre del numero di telefono,
così posso individuare l'account corretto?"
```

Passaggio strutturato del caso

Quando effettui un'escalation, fornisci un riepilogo strutturato perché di solito la persona incaricata non può vedere la trascrizione della conversazione:

```
RIEPILOGO DELL'ESCALATION
==================
Cliente: Jane Doe (n. 12345)
Contatto: jane@example.com | 555-0123
Tipo di problema: richiesta di rimborso — prodotto danneggiato
Ordine: n. 67890 ($149.99, consegnato il 2024-01-20)

Azioni intraprese dall'agente:
1. Identità del cliente verificata.
2. Dettagli dell'ordine e della consegna confermati.
3. Il cliente ha fornito fotografie del danno.
4. L'importo del rimborso ($149.99) supera il limite di approvazione automatica
($100).

Azione consigliata: approvazione manuale per il rimborso completo e la
sostituzione.
Urgenza: standard (il cliente è frustrato, ma non rischia di abbandonare il
servizio).
```

5.3 Propagazione degli errori

Contesto strutturato dell'errore

Quando si verifica un errore, propagalo includendo il contesto completo:

```
{
  "failure_type": "timeout",
  "service": "knowledge_base_search",
  "attempted_query": "politica di reso per dispositivi elettronici oltre $500",
  "timeout_duration_ms": 30000,
  "partial_results": null,
  "alternative_approaches": [
    "Prova a cercare con termini più semplici",
    "Controlla direttamente la sezione FAQ",
    "Chiedi al cliente quale prodotto specifico intende"
```

```
  ],
  "is_retryable": true,
  "retry_after_ms": 5000
}
```

Queste informazioni consentono a Claude di decidere come recuperare in modo efficace.

Antipattern

1. Messaggi di errore generici:

```
{ "error": "Operation failed" }
```

Claude non può recuperare in modo efficace perché non sa cosa sia fallito né perché.

2. Soppressione silenziosa:

```
{ "results": [] }  // Restituito quando il motore di ricerca era in realtà inattivo
```

Claude dirà con sicurezza all'utente «Non ho trovato informazioni al riguardo», quando in realtà «Non sono riuscito a eseguire la ricerca». Sono situazioni fondamentalmente diverse.

3. Interruzione del workflow:

```
Passaggio 1: recupera le informazioni del cliente → Riuscito
Passaggio 2: cerca nella knowledge base → Timeout
Passaggio 3: INTERROMPI L'INTERO WORKFLOW
```

Meglio: continua usando i risultati parziali, segnala la lacuna e proponi alternative.

Errore di accesso e risultato vuoto

Situazione	Stato HTTP	Significato	Risposta dell'agente
Ricerca valida, nessuna corrispondenza	200 + array vuoto	Non esistono contenuti pertinenti	«Non ho trovato articoli corrispondenti»
Servizio di ricerca inattivo	503 / timeout	La ricerca non è stata eseguita	«Al momento non riesco a cercare; provo un altro approccio»

Recupero dagli errori nei subagenti

I subagenti dovrebbero implementare un **recupero locale** per gli errori temporanei:

```
Subagente: cerca "politica di reso"
  → Timeout
  → Nuovo tentativo con attesa progressiva (recupero locale)
  → Riuscito al secondo tentativo
  → Restituisce i risultati al coordinatore
```

Propaga solo gli errori che il subagente non riesce a risolvere. Quando li propaga, includi:

- cosa è stato tentato;
- quali risultati parziali sono stati ottenuti;
- quale errore si è verificato;
- se è possibile riprovare.

5.4 Gestione del contesto nelle codebase ampie

Degrado del contesto nelle sessioni prolungate

Nelle sessioni lunghe, l'affidabilità di Claude peggiora:

- Fa riferimento a «schemi tipici» invece che al codice specifico individuato in precedenza.
- Fornisce risposte incoerenti sullo stesso codice.
- Dimentica le scoperte precedenti.
- Inizia a fare supposizioni invece di verificare.

Strategie di mitigazione

1. File di appunti (scratchpad)

Salva i risultati in file che sopravvivono alla compressione del contesto:

```
# scratchpad.md
## Risultati sull'autenticazione
- Punto di ingresso: src/auth/middleware.ts:23
- Usa JWT con RS256
- Scadenza del token: 1 ora
- Token di aggiornamento: 7 giorni
- Controllo dei ruoli: src/auth/roles.ts:45
- Problema noto: manca un meccanismo per revocare i token
```

Quando il contesto viene compresso, Claude può rileggere gli appunti per recuperare i risultati.

2. Delega ai subagenti

Isola le esplorazioni prolisse nei subagenti:

```
Contesto principale: pulito e concentrato sull'attività
  ↓
```

```
Subagente Explore: legge 50 file ed esegue ricerche approfondite nella codebase
  ↓
Restituisce: riepilogo conciso dei risultati
  ↓
Contesto principale: riceve solo il riepilogo e resta pulito
```

3. Uso proattivo di `/compact`

Durante le sessioni prolungate, compatta il contesto in modo proattivo prima che si degradi. È preferibile compattarlo con un riepilogo strutturato anziché lasciare che la compressione automatica perda informazioni in modo imprevedibile.

4. Modello di recupero dopo un arresto anomalo

Per workflow multiagente di lunga durata:

```
# Ogni agente esporta lo stato in una posizione nota
agent.save_state("/tmp/workflow/agent-1-state.json")

# Il coordinatore mantiene un manifesto
manifest = {
    "agent-1": {"status": "complete", "state": "/tmp/workflow/agent-1-
state.json"},
    "agent-2": {"status": "in_progress", "state": "/tmp/workflow/agent-2-
state.json"},
    "agent-3": {"status": "pending"}
}

# Alla ripresa, il coordinatore carica il manifesto e continua dal punto raggiunto
```

5.5 Workflow di revisione umana

Campionamento casuale stratificato

Non estrarre semplicemente campioni a caso da tutti gli output. Suddividili per rischio o categoria:

```
Categoria A (rischio alto): campiona il 20% degli output
Categoria B (rischio medio): campiona il 10% degli output
Categoria C (rischio basso): campiona il 5% degli output
```

In questo modo concentri la revisione umana sulle aree con maggiore probabilità di errore.

Punteggi di affidabilità per campo

Invece di un unico punteggio di affidabilità per l'intera estrazione, indica il livello per ogni campo:

```
{
  "vendor_name": { "value": "Acme Corp", "confidence": 0.99 },
  "total_amount": { "value": 149.99, "confidence": 0.95 },
  "purchase_order": { "value": "PO-2024-001", "confidence": 0.72 },
  "tax_id": { "value": "12-3456789", "confidence": 0.45 }
}
```

Calibrazione: i punteggi di affidabilità devono essere calibrati usando insiemi di convalida etichettati. Un'affidabilità di 0.9 dovrebbe indicare che il campo è corretto circa il 90% delle volte. Senza calibrazione, questi punteggi non hanno significato.

Accuratezza per tipo di documento

Il problema nascosto:

```
Accuratezza complessiva: 97% ← Sembra ottima!

Breakdown:
  Fatture standard: 99.5% ← Eccellente
  Note manoscritte: 45%  ← Pessima
  Documenti in lingua straniera: 62% ← Scarsa
```

Le metriche aggregate nascondono prestazioni scarse su tipi specifici di documento. Suddividi sempre l'accuratezza per tipo e per campo.

Rapporti distinti per tipo di documento

Riporta le metriche di accuratezza per ogni tipo di documento E per ogni campo al suo interno:

```
Standard Invoices:
  vendor_name: 99.8%
  total_amount: 99.2%
  line_items: 97.5%
  tax_id: 94.1%

Handwritten Notes:
  vendor_name: 85.0%
  total_amount: 42.0% ← Critical failure
  line_items: 38.0% ← Critical failure
```

In questo modo individui dove il sistema deve davvero migliorare, invece di nascondere i problemi dietro un dato aggregato.

5.6 Provenienza delle informazioni

Associazione tra affermazioni e fonti

Quando i subagenti producono risultati, dovrebbero indicarne la provenienza in modo strutturato:

```
{
  "claim": "I ricavi sono aumentati del 15% su base annua nel quarto trimestre
2024",
  "evidence": "Relazione finanziaria del quarto trimestre, pagina 3, paragrafo 2",
  "source_url": "https://example.com/earnings/q4-2024",
  "document_name": "Relazione finanziaria del quarto trimestre 2024",
  "publication_date": "2025-01-15",
  "source_type": "primary"
}
```

Gestione dei dati in conflitto

Quando le fonti non concordano, **annota il conflitto** invece di sceglierne una senza segnalarlo:

```
{
  "field": "Q4 revenue",
  "values": [
    {
      "value": "$2.3B",
      "source": "Comunicato stampa dell'azienda (2025-01-15)",
      "note": "Dati preliminari"
    },
    {
      "value": "$2.28B",
      "source": "Deposito presso la SEC (2025-02-28)",
      "note": "Dati sottoposti ad audit"
    }
  ],
  "resolution": "Il deposito presso la SEC è la fonte autorevole (dati sottoposti
ad audit)",
  "conflict_type": "discrepanza_minore"
}
```

Antipattern: scegliere un valore senza segnalare il disaccordo. Questo nasconde possibili problemi di qualità dei dati.

Dati temporali

Includi le date di pubblicazione e raccolta per evitare ambiguità temporali:

```
{
  "claim": "Il tasso di disoccupazione è del 3.7%",
  "data_as_of": "2024-11-01",
  "source_published": "2024-12-06",
```

```
  "retrieved_on": "2025-01-10"
}
```

Senza date, chi legge potrebbe scambiare due valori diversi del tasso di disoccupazione, riferiti a mesi differenti, per una contraddizione, anche se sono entrambi corretti per i rispettivi periodi.

Qualità dell'output di sintesi

Quando produci una sintesi di ricerca:

1. **Distingui i livelli di certezza:**

- «Ben documentato: i ricavi sono cresciuti del 15% (dato confermato da un deposito SEC e da due rapporti di analisti)».
- «Controverso: le stime della quota di mercato vanno dal 12% al 18%, a seconda della metodologia».
- «Fonte singola: solo un rapporto menziona l'acquisizione pianificata».

1. **Preserva il modo in cui la fonte presenta l'informazione:**

- Non fondere né reinterpretare ciò che affermano le fonti.
- Cita o parafrasa fedelmente l'affermazione originale.
- Riporta le riserve e le qualificazioni indicate dalla fonte.

1. **Scegli il formato in base al tipo di contenuto:**

- Dati finanziari → tabelle con attribuzione delle fonti.
- Notizie ed eventi → testo in ordine cronologico.
- Specifiche tecniche → elenchi strutturati.
- Affermazioni contestate → confronto affiancato.

5.7 Gestione del contesto lato server

Le sezioni 5.1 e 5.4 trattano la gestione del contesto come attività svolta *da te*: blocchi con i fatti del caso, file di appunti, riduzione dell'output e delega ai subagenti. Ora l'API offre tre meccanismi lato server che svolgono parte di questo lavoro. Sono complementari, non alternativi; è importante sapere quale problema risolve ciascuno.

Meccanismo	Funzione	Dove viene conservato lo stato
Modifica del contesto	*Elimina* vecchi risultati degli strumenti o blocchi di thinking prima che il modello li veda	Eliminato definitivamente
Compattazione	*Riassume* il contesto precedente, su richiesta (inviando una richiesta `compaction`) o quando l'input raggiunge una soglia di token	Sostituito nella conversazione da un blocco di riepilogo firmato
Strumento di memoria	Consente a Claude di leggere e scrivere file persistenti	Fuori dalla conversazione; sopravvive a tutto

Modifica del contesto (beta `context-management-2025-06-27`)

Elimina il contenuto; non lo riassume.

```
client.beta.messages.create(
    model="claude-opus-5-5",
    betas=["context-management-2025-06-27"],
    context_management={"edits": [
        {"type": "clear_thinking_20251015",
         "keep": {"type": "thinking_turns", "value": 2}},
        {"type": "clear_tool_uses_20250919",
         "trigger": {"type": "input_tokens", "value": 30000},
         "keep": {"type": "tool_uses", "value": 3},
         "clear_at_least": {"type": "input_tokens", "value": 5000},
         "exclude_tools": ["web_search"]},
    ]},
    tools=[...], messages=[...],
)
```

- `clear_tool_uses_20250919` — elimina i vecchi risultati degli strumenti (`clear_tool_inputs: true` elimina anche i parametri). Attivazione predefinita: 100.000 token di input; valore predefinito di `keep`: 3.
- `clear_thinking_20251015` — elimina i blocchi di thinking. Se usi entrambi, **`clear_thinking` deve comparire per primo**.
- `clear_at_least` protegge la cache del prompt: eliminare una quantità trascurabile invaliderebbe la cache senza alcun vantaggio.
- La risposta indica le operazioni eseguite in `context_management.applied_edits`.
- Con Fable 5.1 e Opus 5.5, la modifica del contesto lato server **non invalida mai i blocchi di thinking**; eliminare gli stessi risultati lato client invece li invaliderebbe (vedi sotto).

Compattazione: su richiesta (beta `compact-2026-09-04`) o al raggiungimento di una soglia (beta `compact-2026-01-12`)

Riassume invece di eliminare. Dal 2026-09-14 sono disponibili due modalità; la documentazione consiglia di **preferire la compattazione su richiesta ogni volta che è disponibile**:

	Su richiesta	**Al raggiungimento di una soglia**
Chi decide quando	Tu, inviando una richiesta	L'API, quando i token di input raggiungono la soglia (predefinita: circa 150k)
Come funziona	Invia la conversazione nello stato corrente (con gli stessi `system` e `tools`) e `compaction: {"type": "summarize"}` al livello principale; la risposta contiene un unico blocco `compaction` firmato, con `stop_reason: "compaction"` e senza risposta testuale	Aggiungi `context_management.edits: [{"type": "compact_20260112"}]` alle richieste normali

	Su richiesta	**Al raggiungimento di una soglia**
Cosa reinviare	A ogni richiesta successiva, inserisci il blocco **per primo** in `messages`, al posto dei messaggi riassunti: un solo blocco, non modificato, insieme all'intestazione beta	L'intero `response.content`, aggiunto come di consueto; l'API elimina ciò che precede il blocco
Turni recenti mantenuti alla lettera	Sì: escludili dalla richiesta di compattazione e inviali dopo il blocco	Solo mettendo in pausa dopo la compattazione e reinserendoli
Esecuzione in background	Sì: la conversazione prosegue mentre viene scritto il riepilogo	No: avviene all'interno della richiesta che ha raggiunto la soglia
I turni conservati mantengono il thinking (Fable 5.1 / Opus 5.5)	Sì, se il riepilogo è stato scritto dall'API e `system` / `tools` non sono cambiati	No: rimuovi o elimina i blocchi
Disponibilità	Fable 5.x, Mythos 5.x, Opus 5.5 / 5 / 4.8 / 4.7 / 4.6, Sonnet 5 / 4.6; Claude API, Claude Platform su AWS, Google Cloud e Foundry — **non Bedrock**	Gli stessi modelli

Dettagli importanti sulla compattazione su richiesta: `instructions` (fino a 16.384 caratteri) *sostituisce* il prompt predefinito per il riepilogo; specifica quindi cosa conservare e chiedi al modello di non chiamare strumenti. La chiamata di riepilogo viene addebitata e indicata in `usage.iterations`, mentre i conteggi dei token al livello principale sono zero. Controlla `stop_reason` prima di cercare il blocco: `max_tokens`, `tool_use`, `refusal` o `end_turn` indicano che non è stato restituito alcun riepilogo; continua quindi la conversazione e riprova più avanti. La richiesta viene rifiutata se termina con una chiamata a uno strumento senza risposta o include `stop_sequences`, `output_config.format`, `tool_choice` forzato, `context_management` o `remaining` di un budget dell'attività. Inoltre, immagini, documenti e URL recuperati nell'intervallo riassunto scompaiono quando vengono sostituiti dal blocco. Anche i messaggi `role: "system"` in quell'intervallo vengono riassunti: ripeti le istruzioni ancora necessarie in un nuovo messaggio `role: "system"` subito dopo il prossimo turno `user`. Il tool runner dell'SDK esegue la sostituzione al posto tuo tramite `compact_before_next_turn()`.

> **Dettaglio d'integrazione fondamentale per la compattazione a soglia:** a ogni turno aggiungi di nuovo a `messages` **l'intero** `response.content`, non solo il testo estratto. Alla richiesta successiva, l'API usa i blocchi di compattazione della risposta per sostituire la cronologia compattata. Estrarre la stringa di testo e aggiungere solo quella distrugge silenziosamente lo stato di compattazione: è un errore subdolo, che potresti notare solo al novantesimo turno.
>
> Anche la compattazione su richiesta presenta due errori silenziosi: lasciare i messaggi riassunti *dopo* il blocco li invia due volte a Claude; omettere il blocco da una richiesta successiva fa sì che Claude non riceva affatto il riepilogo. L'unico caso che genera un errore (`compaction_block_misplaced`) è lasciare i messaggi riassunti *prima* del blocco.

La compattazione lato client dell'SDK (`compaction_control` nel tool runner) è **deprecata** a favore di questi meccanismi.

Thinking preservato: forme di compattazione che restano valide

Fable 5.1 e Opus 5.5 introducono un vincolo assente negli altri meccanismi: i loro blocchi di thinking sono validi solo rispetto alla cronologia esatta che li precede, quindi **modificare i turni precedenti invalida ogni blocco successivo** (Dominio 4 §4.7). La modifica del contesto e la compattazione lato server non contano come modifiche: il controllo confronta la conversazione *così come l'hai inviata* e accetta la sostituzione quando il riepilogo è stato generato dall'API. È l'argomento più forte per spostare la riduzione del contesto sul server. Ecco i diversi schemi e se il thinking dei turni conservati resta valido:

Schema	Regola
Compattazione semplice	Sostituisci l'intera cronologia con un solo riepilogo (il blocco dell'API o un tuo messaggio di riepilogo) e il nuovo turno utente; non riprodurre altro. Nessun blocco di thinking viene mantenuto, quindi non si verificano errori.
Compattazione con coda conservata, generata dall'API (su richiesta)	I turni conservati mantengono thinking valido se valgono tre condizioni: ogni compattazione successiva alla creazione di un blocco è stata eseguita da un modello con thinking preservato; i turni conservati seguono direttamente i messaggi riassunti, senza modifiche, e iniziano con un ruolo diverso da quello dell'ultimo messaggio riassunto (compattare esattamente `messages` di una richiesta già inviata garantisce questo risultato); `system` e gli strumenti `tools` non differiti sono identici nella richiesta di compattazione e in tutte quelle successive.
Compattazione con coda conservata, generata dal client	I turni dell'assistente mantenuti contengono ancora thinking generato rispetto ai turni originali, quindi i blocchi non sono validi. Rimuovi i blocchi `thinking` / `redacted_thinking` (il testo e le chiamate agli strumenti possono restare) oppure invia `prefix_mismatch_behavior: "drop_block"`.
Compattazione in background	Con la compattazione su richiesta, i turni arrivati mentre veniva generato il riepilogo sono conservati e mantengono il thinking se valgono le stesse condizioni. Se in seguito sostituisci la cronologia con un riepilogo scritto dal client, ogni blocco prodotto nel frattempo non è più valido: invia `"drop_block"` a ogni richiesta che contiene ancora thinking precedente alla sostituzione oppure compatta in modo sincrono.

La compattazione non genera errori al momento dell'operazione: il problema si presenta alla prima richiesta successiva che riproduce thinking conservato quando il controllo è attivo. Per impostazione predefinita restituisce 400; con `"drop_block"` i blocchi vengono eliminati e l'operazione è indicata in `input_transformations`. Fai una prova: compatta, conserva un turno con un blocco thinking, invia la richiesta successiva con `prefix_mismatch_behavior: "error"` e verifica di ricevere 200 con `input_transformations` vuoto. Non funziona mai: rimuovere singoli turni dal mezzo della trascrizione, eliminare manualmente i vecchi risultati degli strumenti, inserire un messaggio `role: "system"` tra il blocco e i turni conservati oppure ricostruire `system` / `tools` tra richieste. Per modificare `system` o `tools` in sicurezza, compatta prima l'intera conversazione (senza conservare turni), poi modificali nella richiesta successiva; per cambiare istruzioni o strumenti senza toccare quei campi, aggiungi un messaggio `role:`

`"system"` (Dominio 2 §2.6, Dominio 4 §4.7). Eliminare i blocchi una volta al confine della compattazione costa poco; invalidarli a *ogni* richiesta riavvia ogni volta la cache del prompt.

Strumento di memoria (`memory_20250818`)

È uno strumento lato client: Anthropic ne definisce l'interfaccia, mentre tu implementi le operazioni sui file. Si dichiara come `{"type": "memory_20250818", "name": "memory"}`, senza `input_schema`.

Il suo ruolo in questo dominio è fornire persistenza, rendendo sicura la cancellazione. Se lo combini con la modifica del contesto, Claude riceve l'avviso di salvare in memoria le informazioni importanti *prima* che i risultati dei suoi strumenti vengano eliminati. È l'equivalente a livello API del modello dei file di appunti descritto in §5.4.

Come scegliere

Situazione	**Meccanismo**
Loop agentico con decine di risultati prolissi degli strumenti	Modifica del contesto (`clear_tool_uses`)
Thinking esteso e necessità di mantenere alte le cache hit	Modifica del contesto (`clear_thinking`)
Conversazione lunga di ricerca/analisi di cui occorre conservare la narrazione	Compattazione: su richiesta se disponibile, a soglia quando l'API deve gestirla nelle richieste normali
L'applicazione deve decidere quando compattare, mantenere gli ultimi turni alla lettera o continuare a lavorare durante la generazione del riepilogo	Compattazione su richiesta
Risultati che devono sopravvivere tra sessioni, non solo tra turni	Strumento di memoria
Fatti che non devono mai essere parafrasati	Blocco dei fatti del caso (§5.1): nessun meccanismo protegge i numeri esatti meglio della loro ripetizione

5.8 Limitare e recuperare le esecuzioni lunghe

Budget dell'attività e `max_tokens`

`max_tokens` è un limite che il modello non può vedere; raggiungerlo tronca l'output a metà del ragionamento. Un **budget dell'attività** (`output_config.task_budget`, beta `task-budgets-2026-03-13`, minimo 20.000) è un limite *visibile* al modello, che può così regolare il ritmo e completare il lavoro. È disponibile su Claude Fable 5.1 / Mythos 5.1, Fable 5 / Mythos 5, Opus 5.5, Opus 5 e Opus 4.8/4.7, ma **non** su Sonnet 5, Opus 4.6 o Haiku 4.5. Il budget conteggia ciò che Claude genera più i risultati degli strumenti che legge in questo turno, non l'intera cronologia reinviata. In un loop normale lascia `remaining` non impostato; passalo solo se riscrivi o compatti tu la cronologia e il server non può più ricavare la spesa precedente. Non includerlo mai in una richiesta di compattazione su richiesta né in una richiesta che contiene il relativo blocco

(errore 400). Due note sull'affidabilità: un budget palesemente insufficiente induce Claude a rifiutare, restringere l'ambito o fermarsi in anticipo con un risultato parziale (prima di eseguire altre diagnosi, aumenta il budget); inoltre, il valore del budget viene inserito nel prompt, quindi modificarlo durante l'attività causa un cache miss.

I **budget di sessione** di Managed Agents sono un'altra cosa: limiti rigidi, espressi in valuta e applicati dalla piattaforma a una singola sessione. Il budget dell'attività è consultivo ed espresso in token.

I rifiuti riguardano l'affidabilità, non solo la sicurezza

Un `refusal` arriva con HTTP 200, `stop_reason: "refusal"` e una categoria `stop_details.category`. Il codice che legge `content` senza controllare `stop_reason` tratta il rifiuto come una risposta vuota riuscita: la stessa classe di bug che consiste nel trattare una ricerca non riuscita come un risultato vuoto (§5.3). Nei flussi di produzione, abilita il fallback lato server (`betas=["server-side-fallback-2026-07-01"]`, `fallbacks="default"`) affinché la richiesta venga reindirizzata in base alla categoria invece di essere scartata. Vedi Dominio 1 §1.1. Opus 5.5 ha ampliato l'insieme dei classificatori: a quello per la cybersecurity si aggiunge quello biologico e i prompt che chiedono al modello di riprodurre il proprio ragionamento interno possono essere rifiutati come `reasoning_extraction`. I rifiuti sono quindi un caso da gestire esplicitamente nel modello predefinito, non solo nel modello di punta. Dal 2026-09-24, un rifiuto ricevuto prima di qualsiasi output è **addebitato** se la categoria è `bio`, `frontier_llm` o `reasoning_extraction`; inoltre, ogni rifiuto conta ai fini dei limiti di frequenza, quindi prevedi questi costi. Le protezioni di Mythos 5.1 dipendono dal programma di accesso Project Glasswing: gestisci `refusal` anche con quel modello. Infine, un fallback *da* Fable 5.1 o Opus 5.5 passa di solito a un modello che non può leggere i relativi blocchi di thinking (solo Fable 5.1 / Mythos 5.1 leggono quelli di Opus 5.5), quindi prevedi un turno per pianificare di nuovo.